Cómo funciona la parte visual de ISA Track

Esta guía es para alguien que ya sabe PHP (CodeIgniter) y JavaScript, pero no ha usado Laravel moderno, Vue, Vite ni Inertia. Explica qué hace cada pieza, cómo viaja una pantalla del servidor al navegador, dónde está la arquitectura y qué archivos se usan en desarrollo y cuáles en producción.

1. La idea en una frase

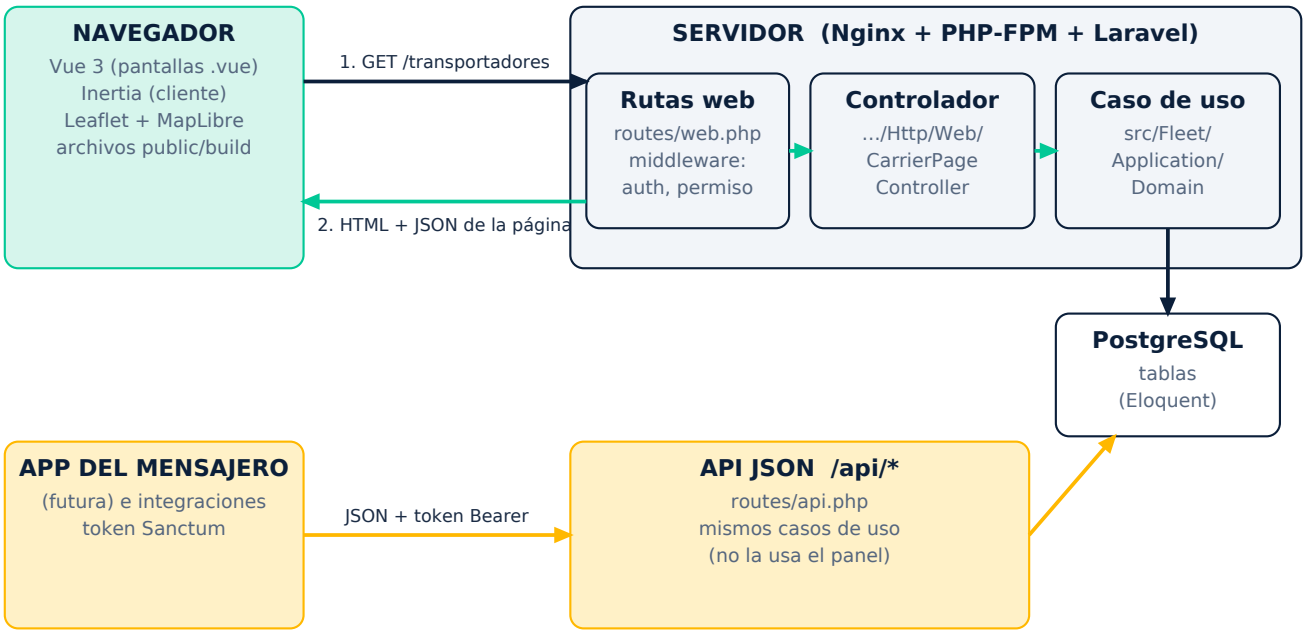
Es como CodeIgniter con vistas, pero las vistas son componentes Vue. El controlador de Laravel consulta los datos y llama `Inertia::render('fleet/pages/CarriersIndex', [...datos])`, igual que `$this->load->view('vista', $data)`. La diferencia: en vez de un HTML armado con PHP, el navegador recibe el nombre de la pantalla y los datos en JSON, y Vue la dibuja. Al navegar no se recarga la página completa: solo cambian los datos y el componente.

CodeIgniter (lo que conoces)	ISA Track (Laravel + Inertia + Vue)	Dónde está
application/config/routes.php	Rutas con su permiso	routes/web.php (panel) · routes/api.php (API)
Controller	Controlador delgado: valida, llama al caso de uso, responde	src/<Módulo>/Infrastructure/Http/Web/
Model	Casos de uso + dominio + repositorio Eloquent	src/<Módulo>/Application, Domain, Infrastructure/Persistence
views/*.php	Componentes Vue (.vue)	resources/js/modules/<módulo>/pages/
$this->load->view(...)	Inertia::render('modulo/pages/Pagina', $props)	en cada controlador web
form_validation	FormRequest / $request->validate()	CarrierRequest, ZoneRequest…
session / flashdata	Igual: sesión y ->with('success', ...)	llega a Vue como props.flash
assets/js sueltos	Vite compila y empaqueta todo	resources/js → public/build

2. Las piezas y para qué sirve cada una

Pieza	Qué es	Cuándo trabaja
Vue 3	Librería de JavaScript para hacer interfaces con componentes. Un archivo .vue junta en un solo lugar la lógica (<script>), el HTML (<template>) y el CSS (<style>). Cuando cambia un dato, Vue actualiza solo esa parte de la pantalla.	En el navegador
TypeScript	JavaScript con tipos (string, number, interfaces). Avisa errores antes de ejecutar, p. ej. si una prop no existe. El navegador nunca lo ve: Vite lo convierte a JavaScript normal.	Solo al compilar
Inertia.js	El "pegamento" entre Laravel y Vue. Del lado PHP: Inertia::render(). Del lado JS: intercepta los clics en <Link> y los envíos de formularios, pide la siguiente página por AJAX y cambia el componente sin recargar.	Servidor y navegador
Vite	El compilador y empaquetador. Toma tus .vue y .ts, los convierte a JavaScript que el navegador entiende, los une, los minifica y les pone un nombre con hash (app-DH_tKjj0.js) para que el navegador no use versiones viejas. En desarrollo además recarga la pantalla al guardar.	Solo al desarrollar o compilar
npm y **package.json**	Es el "composer" de JavaScript. package.json lista las librerías; package-lock.json fija las versiones exactas; node_modules/ es donde se descargan (como vendor/ en PHP).	Solo al instalar o compilar
Leaflet + MapLibre	Librerías del mapa. Leaflet maneja marcadores, polígonos y zoom. MapLibre dibuja el mapa urbano vectorial (calles, comercios, edificios).	En el navegador
Laravel	El framework PHP: rutas, middleware (login, permisos, empresa), validación, base de datos, sesión. Toda la seguridad y las reglas viven aquí.	En el servidor

3. Cómo viaja una pantalla (ejemplo: Transportadores)



El panel NO llama a /api: habla con las rutas web de Laravel, que devuelven la página ya armada con sus datos.

3.1 Primera visita (escribes la dirección o recargas)

- El navegador pide `GET /transportadores`. Laravel busca la ruta en `routes/web.php`:

```
Route::get('/transportadores', [CarrierPageController::class, 'index'])
    ->middleware('permission:carriers.view')->name('panel.carriers.index');
```

- Los middleware revisan en orden: ¿hay sesión? (auth), ¿de qué empresa es? (tenant), ¿tiene el permiso? Si falla alguno, nunca llega al controlador.
- El controlador pide los datos al caso de uso y responde con Inertia:

```
// src/Fleet/Infrastructure/Http/Web/CarrierPageController.php
public function index(ListCarriersHandler $list, TenantContext $tenant): Response
{
    return Inertia::render('fleet/pages/CarriersIndex', [
        'carriers' => $list(),                                  // datos de la base
        'vehicleTypes' => config('isa.fleet.vehicle_labels'),
    ]);
}
```

- Laravel devuelve `resources/views/app.blade.php`: un HTML casi vacío con las etiquetas `<script>` de Vite y un `<div id="app" data-page="{...JSON...}">` que trae el nombre de la página y los datos.
- En el navegador, `resources/js/app.ts` arranca Inertia, lee ese JSON, busca el archivo `modules/fleet/pages/CarriersIndex.vue` y Vue dibuja la pantalla con esos datos como **props**.

3.2 Navegar a otra pantalla (clic en el menú)

- Los enlaces del menú son `<Link href="/zonas">` de Inertia. Al hacer clic, Inertia no recarga la página: hace una petición AJAX a `/zonas` con el encabezado `X-Inertia: true`.
- Laravel ejecuta la misma ruta, el mismo middleware y el mismo controlador, pero al ver ese encabezado responde solo el JSON (componente + props). Inertia cambia el componente y actualiza la URL.

3.3 Enviar un formulario

```
// resources/js/modules/fleet/components/CarrierForm.vue  (resumido)
const form = useForm({ name: '', document_number: '', vehicle_plate: '', photo: null });
form.post('/transportadores');        // envía con la cookie de sesión y el token CSRF
// form.errors.vehicle_plate  -> mensaje de validación que devolvió Laravel
// form.processing            -> true mientras se envía (para deshabilitar el botón)
```

- Laravel valida con `CarrierRequest`. Si falla, hace `redirect back` con los errores, igual que en CodeIgniter, e Inertia los pone en `form.errors` junto a cada campo.
- Si guarda bien, el controlador hace `back()->with('success', 'Transportador registrado.')`. Inertia recarga los datos de la pantalla y el mensaje aparece abajo (componente FlashToast).
- Si una regla de negocio falla (p. ej. "ya existe un mensajero con esa cédula"), el error llega como mensaje general arriba de la página.

> **Datos que todas las pantallas reciben sin pedirlos:** usuario conectado y sus permisos, empresa, menú (ya filtrado por permisos), mensaje flash y configuración del mapa. Se arman en `app/Http/Middleware/HandleInertiaRequests.php` y en Vue se leen con `usePage().props`.

4. ¿Y la API?

Hay dos "puertas" al mismo backend, y ambas usan **los mismos casos de uso** (las reglas de negocio están escritas una sola vez):

	Panel web (lo que ves en el navegador)	API JSON
Rutas	routes/web.php (/pedidos, /rutas, /caja…)	routes/api.php (/api/orders, /api/carriers…)
Quién la usa	Administradores, despacho, gerencia	App del mensajero (futura), integraciones (tienda, Excel, Shopify)
Cómo se autentica	Sesión con cookie httpOnly + token CSRF	Token Bearer de Sanctum (POST /api/auth/login)
Qué responde	Páginas Inertia (componente + props)	Solo JSON
Controladores	src/<Módulo>/Infrastructure/Http/Web/	src/<Módulo>/Infrastructure/Http/

El panel **no llama a /api**. Por eso no hay que guardar tokens en el navegador (una cookie httpOnly no la puede leer JavaScript, así un script malicioso no se la puede robar). La única excepción es el cotizador de zonas, que hace un fetch a una ruta web (/zonas/cotizar) con la misma sesión.

5. Dónde está la arquitectura

El backend usa **arquitectura hexagonal** (puertos y adaptadores) por módulos. Cada módulo de negocio tiene tres capas, y las dependencias solo apuntan hacia adentro:

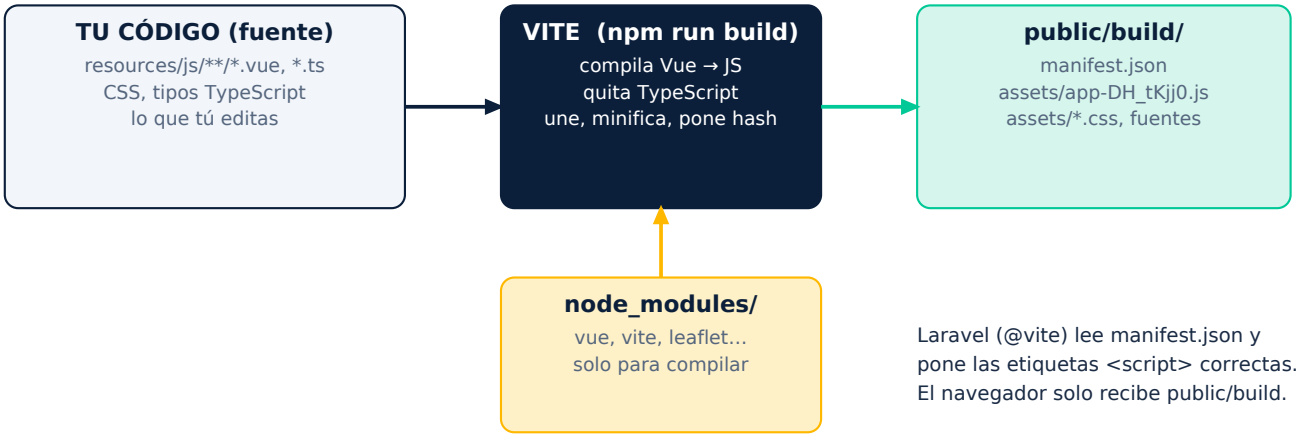
Capa	Qué contiene	Puede usar Laravel
Domain	Reglas del negocio en PHP puro: entidades (Order, Carrier, Zone, DeliveryRoute), valores (GeoPoint, Money), excepciones. Ej.: "una zona necesita 3 puntos", "la ruta no puede pasar la capacidad del vehículo".	No
Application	Casos de uso: un Handler por acción (CreateOrderHandler, PlanRouteHandler…). Orquestan el dominio y piden lo que necesitan por interfaces (puertos).	No
Infrastructure	Lo que habla con el mundo: controladores web y API, modelos Eloquent, repositorios, almacenamiento de fotos, adaptadores entre módulos, ServiceProvider.	Sí

Una prueba automática (`tests/Unit/ArchitectureTest.php`) falla si alguien importa Laravel dentro de Domain o Application. Los módulos están en `src/`: `Shipping` (pedidos), `Fleet` (transportadores), `Zones` (zonas y tarifas), `Routing` (rutas), `Cash` (caja), `Reports` (reportes), `Companies` (empresas) y `Shared` (lo común: permisos, empresa actual, bitácora).

El frontend copia la misma organización por módulos:

```
resources/js/
├── app.ts                       Arranque: busca la página que pide el servidor
├── tracking.ts                  Mapa de la página pública de rastreo (/rastreo/...)
├── types/index.ts               Tipos de los datos que manda el servidor (Carrier, Zone, OrderRow…)
├── modules/                     Una carpeta por módulo, igual que src/
│   ├── fleet/    pages/CarriersIndex.vue      components/CarrierForm.vue
│   ├── zones/    pages/ZonesIndex.vue         components/ZoneMap.vue
│   ├── orders/   pages/OrdersIndex.vue        components/OrderForm.vue, OrderDetail.vue
│   ├── routes/   pages/RoutesIndex.vue        components/RoutePlanner.vue
│   ├── live/  cash/  reports/  users/  dashboard/  auth/
└── shared/                      Lo que usan varias pantallas
    ├── layouts/AppLayout.vue    Menú lateral, barra superior, mensajes
    ├── map/createMap.ts         Crea el mapa (calles urbanas con respaldo)
    ├── map/useMap.ts            Usar un mapa dentro de una pantalla
    ├── ui/                      StatusPill, FieldError, FlashToast
    ├── lib/                     format.ts (dinero, fechas), permissions.ts, icons.ts
    └── styles/                  tokens.css (colores corporativos), base.css, components.css
```

6. Desarrollo vs. producción: qué archivo sirve para qué



Archivo / carpeta	Qué es	Desarrollo	Producción	En GitHub
resources/js/	Tu código fuente (Vue, TypeScript, CSS)	Sí, lo editas	Solo para compilar	Sí
node_modules/	Librerías JS descargadas por npm (como vendor/). Pesa mucho; es normal.	Sí	Solo para compilar; se puede borrar después de npm run build	No (.gitignore)
package.json / package-lock.json	Lista de librerías JS y sus versiones exactas	Sí	Sí (npm ci las usa)	Sí
vite.config.ts, tsconfig.json	Configuración de Vite y TypeScript	Sí	Solo al compilar	Sí
public/build/	Resultado de compilar: JS, CSS y fuentes con nombres con hash (app-DH_tKjj0.js). Es lo que descarga el navegador.	No hace falta con npm run dev	**Sí, obligatorio** (si falta: "Vite manifest not found")	No: se genera en el servidor
public/hot	Archivo temporal que crea npm run dev para decir "usa el servidor de Vite"	Mientras corre npm run dev	**Nunca**; si existe, bórralo	No
vendor/	Librerías PHP (composer)	Sí	Sí (composer install --no-dev)	No
app/, src/, routes/, config/, database/	Código PHP de Laravel	Sí	Sí	Sí
.env	Claves, base de datos, APP_DEBUG	Uno local	Uno propio del servidor	**Nunca**

¿Qué son los archivos raros de public/build/assets? Son tu código ya compilado. Vite les pone un hash en el nombre (CarriersIndex-C8a5tjcE.js): cuando cambias algo, el nombre cambia y el navegador descarga la versión nueva en vez de usar la que tenía en caché. Hay un archivo por pantalla, así el navegador solo descarga la pantalla que abres. El mapa (maplibre-*.js, unos 290 KB comprimido) solo se descarga en las pantallas que tienen mapa. No los edites a mano: se regeneran en cada npm run build.

6.1 Los dos modos de trabajo

	Desarrollo (tu computador)	Producción (servidor)
Comandos	php artisan serve y en otra terminal npm run dev	npm ci y luego npm run build (una vez por actualización)

	Desarrollo (tu computador)	Producción (servidor)
Qué hace Vite	Levanta un servidor (puerto 5173) que entrega los .vue al vuelo; al guardar un archivo la pantalla se actualiza sola, sin perder lo escrito	Compila todo una vez a public/build y se apaga. En producción Vite no está corriendo.
TypeScript	El editor marca los errores mientras escribes	npm run build primero corre vue-tsc: si hay un error de tipos, no compila (así no sube algo roto)
.env	APP_ENV=local, APP_DEBUG=true	APP_ENV=production, APP_DEBUG=false

6.2 Actualizar el servidor (lo que corres después de cada git pull)

```
cd /var/www/isa-track
git pull origin main
composer install --no-dev --optimize-autoloader   # librerías PHP (sin las de pruebas)
npm ci                                            # librerías JS exactas de package-lock.json
npm run build                                     # revisa tipos y genera public/build
php artisan migrate --force                       # cambios en la base de datos
php artisan optimize:clear && php artisan config:cache && php artisan route:cache && php artisan view:cache
# opcional: rm -rf node_modules   (ya no se necesita hasta la próxima compilación)
```

7. Recetas: cómo hacer cambios típicos

7.1 Agregar un campo a una pantalla existente (ej.: "licencia de conducción" del mensajero)

#	Dónde	Qué
1	database/migrations/	Nueva migración: $table->string('license_number')->nullable(); luego php artisan migrate
2	src/Fleet/Domain/CarrierProfile.php	Agregar la propiedad y su regla de validación si tiene
3	src/Fleet/Application/CarrierData.php y CarrierPresenter.php	Recibirla en el comando y devolverla en el arreglo
4	src/Fleet/Infrastructure/Persistence/EloquentCarrierRepository.php	Guardarla y leerla de la tabla
5	src/Fleet/Infrastructure/Http/CarrierRequest.php	Regla de formato: 'license_number' => ['nullable', 'string', 'max:30']
6	resources/js/types/index.ts	license_number: string | null; en la interfaz Carrier
7	resources/js/modules/fleet/components/CarrierForm.vue	Campo en useForm({...}) y un <input v-model="form.license_number">
8	tests/Feature/	Una prueba que lo guarde y lo lea; php artisan test

7.2 Agregar una pantalla nueva (ej.: "Clientes")

- Controlador web que termine en `Inertia::render('customers/pages/CustomersIndex', [...])`.
- Ruta en `routes/web.php` dentro del grupo con login, con su `->middleware('permission:customers.view')`.
- Permiso en `config/isa.php` (catálogo y roles) y entrada en `'navigation'` para que salga en el menú.
- Archivo `resources/js/modules/customers/pages/CustomersIndex.vue` que use `<AppLayout title="Clientes">` y reciba los datos con `defineProps`.

7.3 Anatomía mínima de una página .vue

```
<script setup lang="ts">
import AppLayout from '@/shared/layouts/AppLayout.vue';
import { usePermissions } from '@/shared/lib/permissions';
import { router } from '@inertiajs/vue3';

const props = defineProps<{ customers: { id: string; name: string }[] }>();   // lo que mandó el controlador
const { can } = usePermissions();                                              // solo para mostrar/ocultar botones
const remove = (id: string) => router.delete(`/clientes/${id}`, { preserveScroll: true });
</script>

<template>
  <AppLayout title="Clientes">
    <ul>
      <li v-for="c in customers" :key="c.id">
        {{ c.name }}  <!-- Vue escapa el texto: no hay inyección de HTML -->
        <button v-if="can('customers.manage')" @click="remove(c.id)">Eliminar</button>
      </li>
    </ul>
  </AppLayout>
</template>
```

Regla de oro de seguridad: lo que se oculta en Vue con can() es solo comodidad visual. La protección real es el `->middleware('permission:...')` de la ruta en Laravel, que se revisa en cada petición, y el filtro por empresa en los repositorios.

8. Seguridad que ya trae el panel

- **Sesión de servidor** con cookie httpOnly y protección CSRF (Laravel la revisa en cada POST, PUT o DELETE; Inertia envía el token solo).
- **Login** con máximo 5 intentos por minuto, regeneración de sesión y usuarios inactivos bloqueados. Cada intento queda en la bitácora.
- **Content-Security-Policy**: el navegador solo ejecuta los scripts de public/build firmados con un nonce. No se cargan librerías de CDNs externos.
- **Permisos y empresa** verificados en el servidor en cada ruta; un usuario no puede ver datos de otra empresa.
- **Archivos**: fotos validadas por su contenido real (JPG, PNG o WebP) y servidas solo con login. El CSV para Excel se protege contra inyección de fórmulas.
- **Con HTTPS** se activan además el historial cifrado y los encabezados HSTS y COOP (sin HTTPS el navegador no los permite).

Archivos para leer primero, en este orden: routes/web.php → src/Fleet/Infrastructure/Http/Web/CarrierPageController.php → resources/js/modules/fleet/pages/CarriersIndex.vue → resources/js/shared/layouts/AppLayout.vue → app/Http/Middleware/HandleInertiaRequests.php.